Conjunto de instrucciones

Instrucciones de Transferencia de datos y Aritméticas

Conjunto de Instrucciones del i386

- Se pueden dividir en 5 categorías:
  - Transferencia de datos
  - Aritméticas
  - Operaciones Lógicas
  - Corrimientos y Rotaciones
  - Control del Programa

Instrucciones de transferencia de datos

XCHG

- Intercambia los valores de los operandos.

- Si EAX = 0xE7027193 y EBX = 0x1A894032
  - XCHG EAX, EBX
- Al ejecutar la instrucción:
- EAX = 0x1A894032 y EBX = 0xE7027193
- También se puede operar sobre direcciones de memoria
  - XCHG DL, [EBX]

Restricciones

- 1. No se transfiere entre dos direcciones de memoria.

INVALIDO:

XCHG [1234], [39AC]

Restricciones

- 2. Los operados deben ser del mismo tamaño.

INVALIDO:

XCHG EBX, AX

Restricciones

- 3. No es válido con registros de segmento.

INVALIDO:

XCHG DS, AX

IN y OUT

- **IN:**
- Lee un dato de un dispositivo de E/S y lo almacena en el acumulador.

- **OUT:**
- Transfiere un dato del acumulador a un puerto de E/S.

Tamaño del dato

- Lo determina el registro del acumulador que utilicemos.
- AL = 8 bits
- AX = 16 bits
- EAX = 32 bits

- IN AL, *puerto*
- IN AX, *puerto*
- IN EAX, *puerto*

- OUT *puerto*, AL
- OUT *puerto*, AX
- OUT *puerto*, EAX

Dirección del puerto

- Se puede especificar utilizando una dirección fija (8 bits) o una dirección variable (16 bits) utilizando el registro DX.

- IN AL, *80h*
- IN AX, *DX*

- OUT *4Fh*, AX
- OUT *DX*, EAX

PUSH y POP

- **PUSH:**
- Inserta un dato de 16/32 bits a la pila.

- **POP:**
- Remueve un dato de 16/32 bits de la pila.

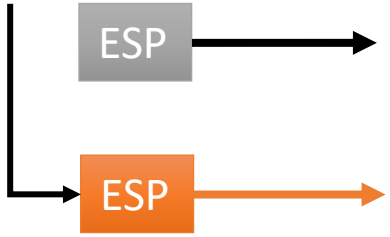
PUSH (16 bits)

EAX = C3B78038

PUSH AX

Antes de ejecutar la instrucción ESP apunta a la dirección 0x19F.

Después de ejecutar la instrucción, ESP apunta a la dirección 0x19D.



Dirección	Contenido
19F	
19E	80
19D	38
19C	
19B	

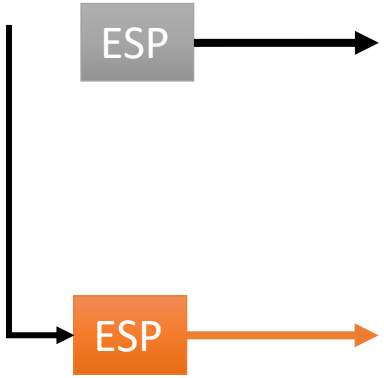
PUSH (32 bits)

EAX = C3B78038

PUSH EAX

Antes de ejecutar la instrucción ESP apunta a la dirección 0x19F.

Después de ejecutar la instrucción, ESP apunta a la dirección 0x19B.



Dirección	Contenido
19F	
19E	C3
19D	B7
19C	80
19B	38

Otras formas de PUSH

- PUSH WORD 4567h: Inserta a la pila el valor inmediato 4567h.
- PUSH DWORD 154567h: Inserta a la pila el valor inmediato 154567h.
- PUSH WORD [ECX]: Inserta a la pila el valor almacenado en Mem[ECX].
- PUSH DWORD [ECX + ESI*4]: Inserta a la pila el valor almacenado en Mem[ECX + ESI*4].

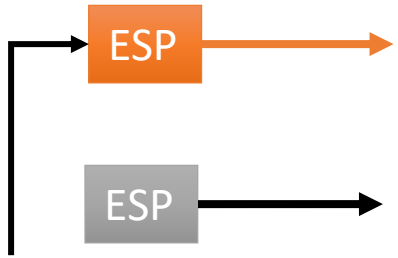
POP (16 bits)

EAX = C3B78038

POP AX

Antes de ejecutar la instrucción ESP apunta a la dirección 0x19D.

Después de ejecutar la instrucción, ESP apunta a la dirección 0x19F.



Dirección	Contenido
19F	
19E	47
19D	F1
19C	
19B	

El valor F1 se almacena en AL y el valor 47 se almacena en AH.

Al final:

EAX = C3B747F1

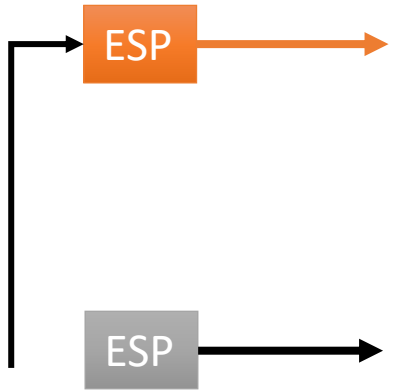
POP (32 bits)

EAX = C3B78038

POP EAX

Antes de ejecutar la instrucción ESP apunta a la dirección 0x19B.

Después de ejecutar la instrucción, ESP apunta a la dirección 0x19F.



Dirección	Contenido
19F	
19E	47
19D	F1
19C	32
19B	D7

El valor D7 se almacena en AL, el valor 32 se almacena en AH, el valor F1 se almacena en los bits 16-23, y el valor 47 se almacena en los bits 24-31.

Al final:

EAX = 47F132D7

Otras formas de POP

- POP WORD [ECX]: Remueve de la pila 16 bits y los almacena en Mem[ECX].

- PUSH DWORD [ECX + ESI*4]: Remueve de la pila 32 bits y los almacena en Mem[ECX + ESI*4]

Restricciones

- 1. Modificar el registro CS

INVALIDO:

POP CS

PUSHF y POPF

- Mete o remueve el registro de banderas a la pila

EFLAGS

31-18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reservados	VM	RF		NT	IO PL	IO PL	O	D	I	T	S	Z		A		P		C

Pila

Figura 1: Registro de banderas. PUSHF mete los bits 0-15 del registro de banderas a la pila. POPF remueve 2 bytes de la pila y los almacena en EFLAGS.

PUSHFD y POPFD

- Mete o remueve el registro de banderas a la pila

EFLAGS

31-18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reservados	VM	RF		NT	IO PL	IO PL	O	D	I	T	S	Z		A		P		C

Pila

Figura 2: Registro de banderas. PUSHFD mete los bits 0-31 del registro de banderas a la pila. POPF remueve 4 bytes de la pila y los almacena en EFLAGS.

LAHF

- Carga el byte menos significativo del registro de banderas en AH.

EFLAGS

31-18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reservados	VM	RF		NT	IO PL	IO PL	O	D	I	T	S	Z		A		P		C

AH

Figura 3: Representación de como los bits 0-7 de EFLAGS son almacenados en AH.

SAHF

- Almacena el valor de AH en el byte menos significativo del registro de banderas.

EFLAGS

31-18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reservados	VM	RF		NT	IO PL	IO PL	O	D	I	T	S	Z		A		P		C

AH

Figura 3: Representación de como AH es almacenado en los bits 0-7 de EFLAGS.

Instrucciones Aritméticas

ADD

- Suma los operandos y almacena el resultado en el primer operando.
  - ADD EAX, EBX
  - ADD CX, 56h
  - ADD DL, [EBX]
  - ADD [EBP + EBX*4 +45h], SI

EFLAGS

31-18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reservados	VM	RF		NT	IO PL	IO PL	O	D	I	T	S	Z		A		P		C

Figura 4: La instrucción ADD puede modificar los bits de Sobreflujo, Signo, Zero, Auxiliar, Paridad, Acarreo del registro de banderas.

Restricciones

- 1. No se permite con registros de segment.

INVALIDO:

ADD SS, 5

ADC

- Realiza la operación de suma entre los operandos, sumándole el bit de acarreo del registro de banderas.
  - ADC EAX, EBX se puede traducir como EAX = EAX + EBX + bit C.
    - ADC CX, 56h se puede traducir como CX = CX + 56h + bit C.
  - ADC DL, [EBX] se puede traducir como DL = DL + Mem[EBX] + bit C.

Ejemplo ADC

- Si se quieren sumar dos números de 64 bits.
- Suponer que EBX es los 32 bits más significativos y EAX los 32 bits menos significativos de un número de 64 bits.
- EDX y ECX son los 32 bits más significativos y menos significativos, respectivamente, de otro número de 64 bits.
- Para calcular la suma de estos 2 números de 64 bits se pueden sumar las partes menos significativas con la instrucción ADD y las partes más significativas con la instrucción ADC.

Ejemplo ADC

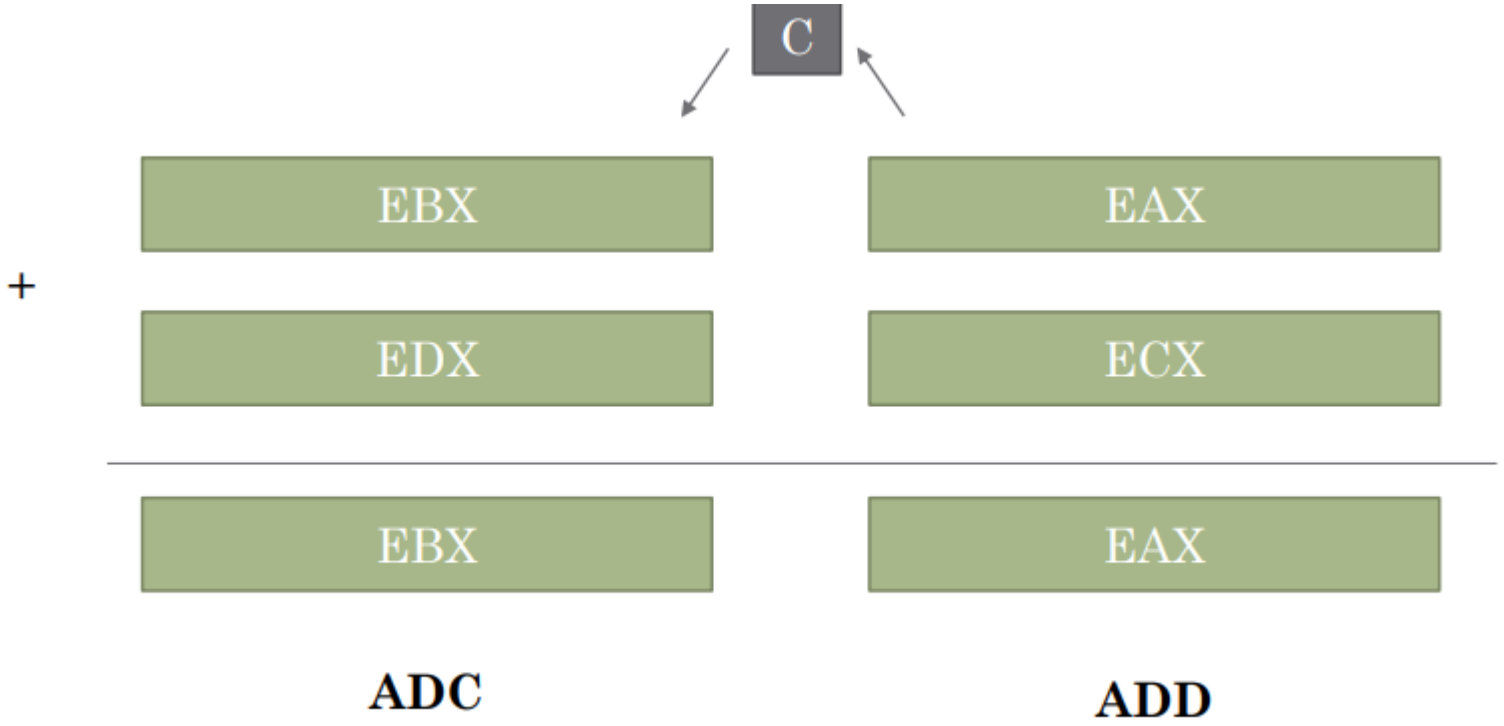


Figura 5: Representación de la suma de dos números de 64 bits. Se empela la instrucción ADD EAX, ECX para los 32 bits menos significativos. Y la instrucción ADC EBX, EDX para los 4 bytes más significativos.

INC

- Incrementa en 1 el operando.
    - INC EAX se puede traducir como EAX = EAX + 1
    - INC BYTE[EBX] se puede traducir como Mem[EBX] += 1
    - INC DWORD [EBP + EBX] se puede traducir como MempEBP + EBX] += 1

INC

- Suponer que Mem[EBP + EBX] = 1F98FEFFh
    - INC BYTE [EBP + EBX] da como resultado 1F98FE00
    - INC WORD [EBP + EBX] da como resultado 1F98FF00

SUB

- Resta los operandos y almacena el resultado en el primer operando.
  - SUB EAX, EBX
  - SUB CX, 56h
  - SUB DL, [EBX]
  - SUB [EBP + EBX*4 + 45h], SI

SBB

- Realiza la operación de resta entre los operandos, restando el bit de acarreo del registro de banderas.
    - SBB EAX, EBX se puede traducir como EAX = EAX - EBX - bit C.
    - SBB CX, 56h se puede traducir como CX = CX - 56h - bit C.
    - SBB DL, [EBX] se puede traducir como DL = DL - Mem[EBX] - bit C.

Ejemplo SBB

- Si se quieren restar dos números de 64 bits.
- Suponer que EBX es los 32 bits más significativos y EAX los 32 bits menos significativos de un número de 64 bits.
- EDX y ECX son los 32 bits más significativos y menos significativos, respectivamente, de otro número de 64 bits.
- Para calcular la resta de estos 2 números de 64 bits se pueden restar las partes más significativas con la instrucción SBB y las partes menos significativas con la instrucción SUB.

Ejemplo SBB

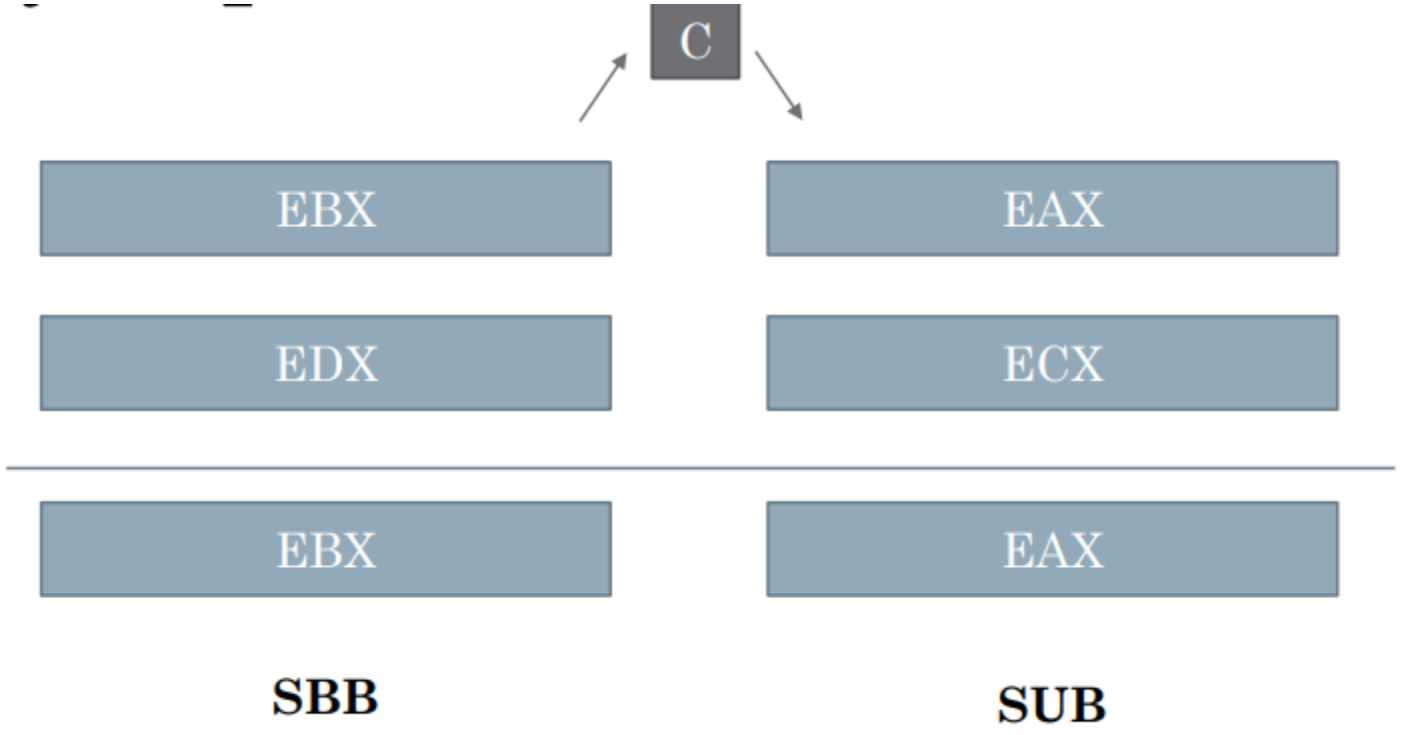


Figura 6: Representación de la resta de dos números de 64 bits. Se empela la instrucción SUB EAX, ECX para los 32 bits menos significativos. Y la instrucción SBB EBX, EDX para los 4 bytes más significativos.

DEC

- Decrementa en 1 el operando.
  - DEC EAX se puede traducir como EAX = EAX - 1
  - DEC BYTE[EBX] se puede traducir como Mem[EBX] -= 1
  - DEC DWORD [EBP + EBX] se puede traducir como MempEBP + EBX] -= 1

DEC

- Suponer que Mem[EBP + EBX] = 1F980100h
    - DEC BYTE [EBP + EBX] da como resultado 1F9801FF
    - DEC WORD [EBP + EBX] da como resultado 1F9800FF

INC y DEC

- Actualizan las banderas excepto **C**(Acarreo**).**

MUL

- Multiplica el operando por AL, AX o EAX.
  - MUL BL se puede traducir como AX = AL*BL
  - MUL DI se puede traducir como DX:AX = AX*DI
  - MUL ECX se puede traducir como EDX:EAX = EAX*ECX
  - MUL Word[EBP] se puede traducir como DX:AX = AX*Mem[EBP]

IMUL

- Sintaxis de la instrucción
  - IMUL reg
  - IMUL mem
  - IMUL reg, reg, immediate
  - IMUL reg, mem, immediate
  - IMUL reg, immediate
  - IMUL reg, reg
  - IMUL reg, mem

IMUL

- IMUL reg / IMUL mem
  - IMUL BL puede traducirse como AX = AL*BL
  - IMUL DI puede traducirse como DX:AX = AX*DI
  - IMUL ECX puede traducirse como EDX:EAX = EAX*ECX
  - IMUL Word[EBP] puede traducirse como DX:AX= AX*Mem[EBP]

IMUL

- IMUL reg,reg,immediate / IMUL reg,mem,immediate
    - IMUL BX, CX, 1234H puede traducirse como BX = CX*1234H
- IMUL EDX, ESI, F50284H puede traducirse como EDX = ESI*F50284H
- IMUL DI, [ECX], 1325H puede traducirse como DI = Mem[ECX]*1325H
- No se pueden utilizar registros o datos de memoria de 8 bits.

IMUL

- IMUL reg, immediate / IMUL reg, reg / IMUL reg, mem
    - IMUL BX, 1234H puede traducirse como BX = BX*1234h
    - IMUL EDX, F50284h puede traducirse como EDX = EDX*F50284h
    - IMUL EBX, EDI puede traducirse como EBX = EBX*EDI
    - IMUL BP, [EBX] puede traducirse como BP = BP*Mem[EBP]
- No es válido utilizar registros de 8 bits.

DIV e IDIV (División 8 bits)

- AL = AX / operando
- AH = AX % operando

- DIV BL se puede traducir como AL = AX/BL y AH = AX%BL
- IDIV CL se puede traducir como AL = AX/CL y AH = AX%CL
- DIV byte[ESI] se puede traducir como AL = AX/Mem[ESI] y AH = AX%Mem[ESI]

DIV e IDIV (División 16 bits)

- AX = DX:AX / operando
- DX = DX:AX % operando

- DIV BX se puede traducir como AX = DX:AX/BX y DX = DX:AX%BX
- DIV CX se puede traducir como AX = DX:AX/CX y DX = DX:AX%CX
- DIV word[ESI] se puede traducir como

  AX = DX:AX/Mem[ESI] y DX = DX:AX % Mem[ESI]

DIV e IDIV (División 32 bits)

- EAX = EDX:EAX / operando
- EDX = EDX:EAX % operando

- DIV EBX se puede traducir como EAX = EDX:EAX/EBX y EDX = EDX:EAX%EBX
- DIV ECX se puede traducir como EAX = EDX:EAX/ECX y EDX = EDX:EAX%ECX
- DIV dword[ESI] se puede traducir como

  EAX = EDX:EAX/Mem[ESI] y EDX = EDX:EAX % Mem[ESI]

LEA

- Guarda en el primer operando la dirección efectiva del segundo operando.

- LEA AX, DATA se puede traducir como AX = Dirección de DATA

- LEA ECX, [EBP+3] se puede traducir como ECX = EBP+3